ZQ 2

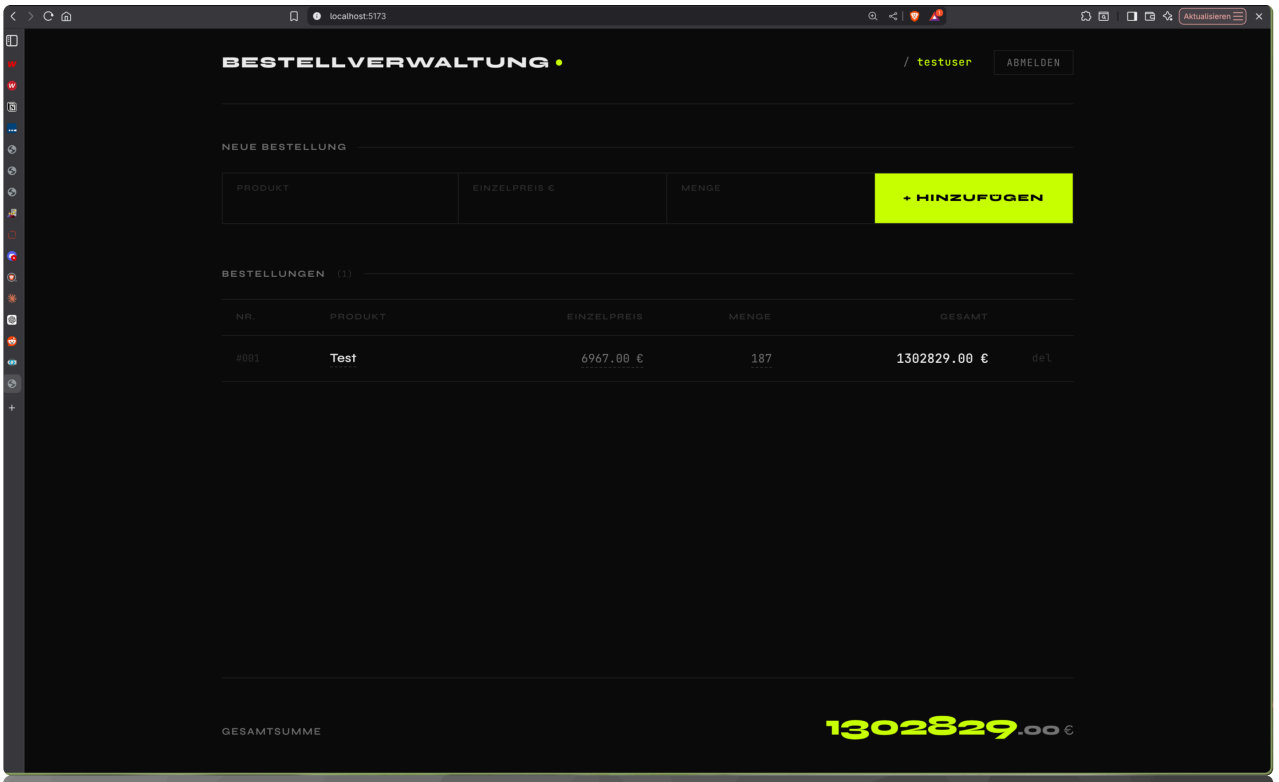
Technische Dokumentation – Bestellverwaltung

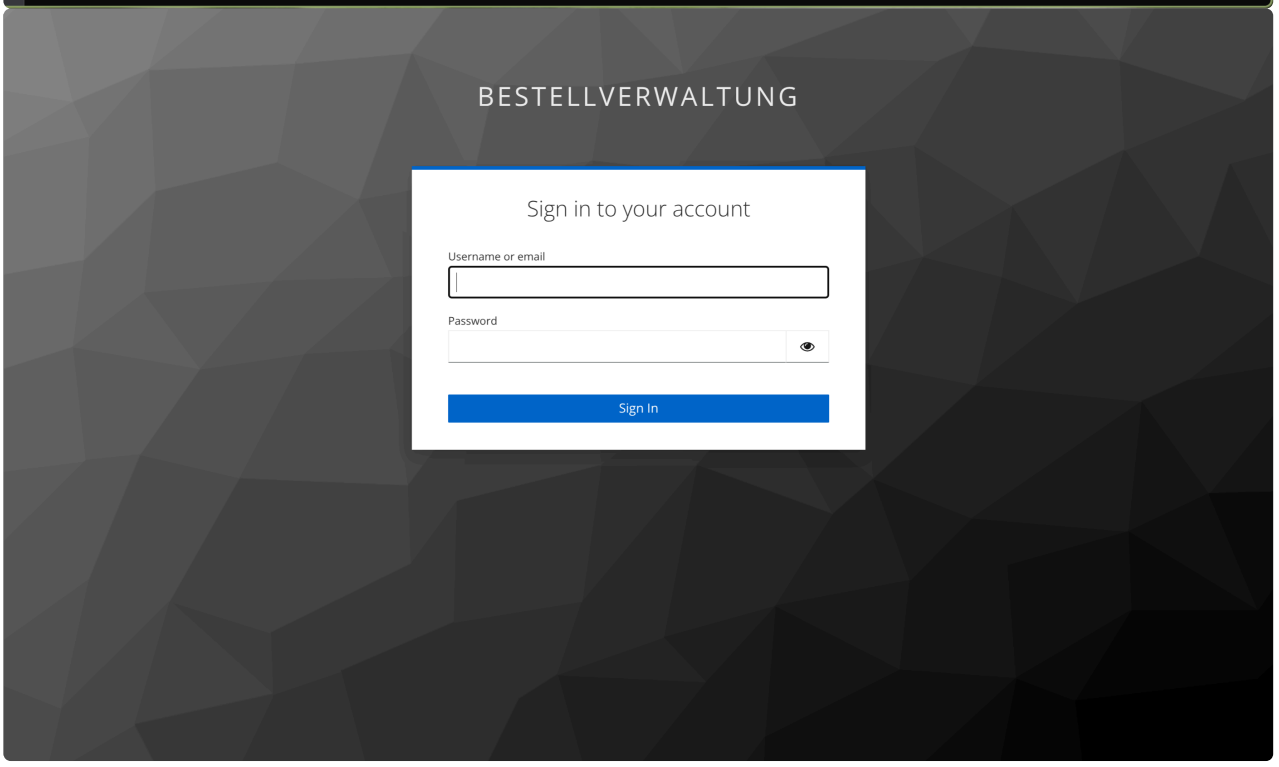
Fach: ZQ
Thema: Prompting – KI als Entwicklungsassistent im Berufsalltag**Datum:** 02.03.202

Phase 1 – Programmbeschreibung

Die Bestellverwaltung ist eine webbasierte Vollstack-Anwendung zur Erfassung,
Bearbeitung und Verwaltung von Bestellungen. Der Zugriff ist durch eine Authentifizierung über Keycloak abgesichert – nur angemeldete Benutzer können die Anwendung nutzen.

Komponente	Technologie	Port
Backend	Springboot	8081
Frontend	React18 + Vite	5173
Auth	Keycloak 24	8080
Datenbank	H2	





Bestellverwaltung – Implementierungsplan

Stack

- **Backend:** Spring Boot 3 (Java 17), Spring Security, Spring Data JPA, H2
- **Auth:** Keycloak (Docker Container)
- **Frontend:** React (Vite) + Axios
- **Infra:** Docker Compose (Keycloak)

Projektstruktur

```
bestellverwaltung/
├── docker-compose.yml # Keycloak Container
├── keycloak/
│ └── realm-export.json # Keycloak Realm Konfiguration
(vorkonfiguriert)
├── backend/ # Spring Boot Projekt
│ ├── pom.xml
│ └── src/main/
│ ├── java/.../
│ │ ├── controller/OrderController.java
│ │ ├── model/Order.java
│ │ ├── repository/OrderRepository.java
│ │ ├── service/OrderService.java
│ │ └── config/SecurityConfig.java
│ └── resources/
│ └── application.yml
└── frontend/ # React (Vite)
├── package.json
├── src/
│ ├── main.jsx
│ ├── App.jsx
│ ├── keycloak.js # Keycloak-JS Adapter
│ └── components/
│ ├── OrderList.jsx
│ └── OrderForm.jsx
└── index.html
```

Schritte

1. `docker-compose.yml` mit Keycloak erstellen
2. Keycloak Realm JSON vorkonfigurieren (Realm, Client, Testuser)
3. Spring Boot Backend:
   - Order Entity + Repository + Service + Controller (CRUD)
   - SecurityConfig mit JWT/Keycloak Resource Server
   - application.yml konfigurieren
4. React Frontend:
   - Vite-Projekt initialisieren
   - Keycloak-JS Adapter einbinden

- OrderList + OrderForm Komponenten
- Axios mit Bearer Token

Phase 2 – Debugging

Fehlerhafter Code (Ausgangslage)

```
@PostMapping

public ResponseEntity<Order> create(@RequestBody Order order) {
String menge = order.getMenge(); // Fehler 1
String preis = order.getEinzelpreis(); // Fehler 2
int gesamtpreis = menge * preis; // Fehler 3
order.setGesamtpreis(gesamtpreis);
return ResponseEntity.ok(repository.save(order));
}
```

1. Analyse des Problems

Der Code enthält drei zusammenhängende Fehler:

| # | Zeile | Problem |

|---|---|---|

| 1 | `String menge = order.getMenge()` | `getMenge()` gibt `int` zurück – Zuweisung an `String` ist falsch |

| 2 | `String preis = ...getEinzelpreis()` | `getEinzelpreis()` gibt `double` zurück – ebenfalls falscher Typ |

| 3 | `int gesamtpreis = menge * preis` | Strings können nicht multipliziert werden – Compilerfehler |

NR	Zeile	Problem
1	`String menge = order.getMenge()`	`getMenge()` gibt `int` zurück – Zuweisung an `String` ist falsch
2	`String preis = ...getEinzelpreis()`	`getEinzelpreis()` gibt `double` zurück – ebenfalls falscher Typ
3	`int gesamtpreis = menge * preis`	Strings können nicht multipliziert werden – Compilerfehler

2. Warum tritt der Fehler auf?

Java ist eine **streng typisierte** Sprache. Jede Variable hat einen festen Datentyp,
der zur Compilezeit geprüft wird. `String` und `int` / `double` sind grundverschieden:

Typ	Art	Beispielwert	Rechenoperationen
`String`	Zeichenkette	`"42"`	Nicht möglich (`*`, `+` = Verkettung)
`int`	Ganzzahl	`42`	Voll möglich
`double`	Dezimalzahl	`9.99`	Voll möglich

Der Operator `*` funktioniert nur mit numerischen Typen. Auf zwei `String`-Variablen
angewendet, wirft der Java-Compiler sofort einen Fehler:
`error: bad operand types for binary operator '*'`

3. Unterschied String vs. Integer

- **String** speichert Text als Folge von Zeichen. Die Zahl `"5"` ist für Java nur ein Zeichen – kein Zahlenwert. `"5" * "3"` ist in Java nicht definiert.
- **Integer / int** speichert eine ganze Zahl als Binärwert im Speicher. `5 * 3 = 15` funktioniert direkt.
- Eine Umwandlung ist möglich: `Integer.parseInt("5")` → `5`, aber im vorliegenden Code wäre das unnötig – die Getter liefern bereits die

richtigen numerischen Typen.

4. Korrigierter Code

```
@PostMapping
public ResponseEntity<Order> create(@Valid @RequestBody Order
order) {
    // Korrekte Typen direkt aus der Entity verwenden
    int menge = order.getMenge(); // int, kein String
    double preis = order.getEinzelpreis(); // double, kein
String
    // Gesamtpreis korrekt berechnen – wird jedoch besser in
@PrePersist gemacht
    double gesamtpreis = menge * preis;
    order.setGesamtpreis(gesamtpreis);
    return ResponseEntity.ok(repository.save(order));
}
```

Eigene Verbesserung gegenüber KI-Vorschlag:

Die Berechnung wurde aus dem Controller in die Entity (`@PrePersist` / `@PreUpdate`) verschoben, sodass der Gesamtpreis bei jeder Speicheroperation – egal von wo –
automatisch korrekt berechnet wird. Das entspricht dem Prinzip der Schichtentrennung.

Phase 3 – Technische Dokumentation

Funktionsweise

Authentifizierung (Keycloak)

Keycloak läuft als Docker-Container und übernimmt die Benutzerverwaltung:

1. Browser öffnet `localhost:5173`

2. Keycloak-JS erkennt fehlenden Token → Weiterleitung zur Loginseite
3. Nach Login wird ein **JWT-Token** ausgestellt
4. Token wird bei jedem API-Request im `Authorization: Bearer`-Header mitgesendet
5. Backend prüft Token als OAuth2 Resource Server gegen Keycloak
6. Läuft Token ab, erneuert `keycloak.updateToken(30)` ihn automatisch

Backend (Spring Boot)

- REST-API unter `/api/orders`
- Eingabevalidierung via `jakarta.validation` (`@NotBlank`, `@DecimalMin`, `@Min`)
- Gesamtpreis wird automatisch in der Entity berechnet (`@PrePersist` / `@PreUpdate`)
- Daten in H2 In-Memory-Datenbank (gehen bei Neustart verloren)

Frontend (React)

- Alle Bestellungen in übersichtlicher Tabelle mit dunklem Design
- Neue Bestellungen über integriertes Formular erfassen
- **Inline-Bearbeitung:** Doppelklick auf Produktname, Preis oder Menge → direkt editierbar
- Enter = Speichern | Escape = Abbrechen | ungültige Werte werden automatisch verworfen
- Gesamtsumme aller Bestellungen wird live am Seitenende angezeigt

Verwendete Variablen / Datenfelder

Order (Backend-Entität, `Order.java`)

Feld	Typ	Beschreibung	Validierung
`id`	`Long`	Automatisch generierter Primärschlüssel	–
`produktname`	`String`	Name des bestellten Produkts	`@NotBlank`
`einzelpreis`	`double`	Preis pro Einheit in Euro	`@DecimalMin("0.01")`

Feld	Typ	Beschreibung	Validierung
`menge`	`int`	Bestellte Stückzahl	`@Min(1)`
`gesamtpreis`	`double`	Automatisch: `einzelpreis × menge`	via `@PrePersist`

Frontend-Zustandsvariablen

App.jsx

Variable	Typ	Beschreibung
`orders`	`Array`	Liste aller Bestellungen vom Backend
`summe`	`number`	Gesamtsumme aller Bestellungen in Euro

OrderForm.jsx

Variable	Typ	Beschreibung
`form`	`Object`	Eingabefelder: `produktname`, `einzelpreis`, `menge`
`fehler`	`String`	Fehlermeldung bei ungültiger Eingabe
`loading`	`boolean`	Sperrt Button während des API-Requests
`focused`	`String`	Aktuell fokussiertes Feld (visuelles Feedback)

OrderList.jsx – EditableCell

Variable	Typ	Beschreibung
`editing`	`boolean`	Gibt an ob das Feld gerade bearbeitet wird
`draft`	`String`	Temporärer Wert während der Eingabe

Programmablauf

```
Browser öffnet http://localhost:5173
│
▼
Keycloak: Kein Token → Loginseite
│
▼ (testuser / password)
JWT-Token ausgestellt
│
▼
React App lädt — Axios-Interceptor registriert
│
▼
GET /api/orders → Tabelle befüllen
GET /api/orders/summe → Gesamtsumme anzeigen
│
┌────┬──────────────────┐
▼ ▼ ▼
NEU BEARBEITEN LÖSCHEN
│ │ │
│ Doppelklick DELETE /api/orders/{id}
│ auf Tabellenzelle │
│ │ 204 → Liste neu laden
│ Enter → PUT
│ Esc → Revert
│ │
▼ ▼
POST / PUT /api/orders[/{id}]
│
▼
Validierung (Backend)
├─ Fehler → HTTP 400 → Fehlermeldung
└─ OK → @PrePersist berechnet Gesamtpreis
→ H2 speichern → HTTP 200
→ Liste + Summe neu laden
```

REST-API Übersicht

Methode	Endpunkt	Beschreibung	Auth
GET	`/api/orders`	Alle Bestellungen abrufen	JWT
POST	`/api/orders`	Neue Bestellung erstellen	JWT

Methode	Endpunkt	Beschreibung	Auth
PUT	`/api/orders/{id}`	Bestellung aktualisieren	JWT
DELETE	`/api/orders/{id}`	Bestellung löschen	JWT
GET	`/api/orders/summe`	Gesamtsumme aller Bestellungen	JWT

Beispiel PUT-Request / Response:

```
// Request Body
{ "produktname": "Laptop", "einzelpreis": 899.99, "menge": 2 }
// Response
{ "id": 1, "produktname": "Laptop", "einzelpreis": 899.99,
"menge": 2, "gesamtpreis": 1799.98 }
```

Projektstruktur

```
bestellverwaltung/
├── docker-compose.yml – Keycloak Container
├── keycloak/
│ └── realm-export.json – Realm, Client, Testuser
├── backend/
│ ├── pom.xml – Maven (Java 17, Spring Boot 3)
│ └── src/main/java/.../
│ ├── config/SecurityConfig – JWT + CORS
│ ├── controller/OrderController – REST-Endpunkte
(GET/POST/PUT/DELETE)
│ ├── service/OrderService – Geschäftslogik
│ ├── repository/OrderRepository – JPA + Summen-Query
│ └── model/Order – Entität + @PrePersist Berechnung
└── frontend/
└── src/
├── keycloak.js – Adapter-Konfiguration
├── main.jsx – Login-Required vor React-Start
├── App.jsx – Axios-Interceptor, State-Management
└── components/
├── OrderForm.jsx – Neues Bestellformular
└── OrderList.jsx – Tabelle + Inline-Editing
```

Hinweise zur Erweiterung

Erweiterung	Umsetzung
Persistente Datenbank	H2 → PostgreSQL, `spring.datasource.*` in `application.yml`
Rollen-basierter Zugriff	`@PreAuthorize("hasRole('ADMIN')")` + Keycloak-Rolle zuweisen
Produktkatalog	Neue Entität `Product`, Fremdschlüssel in `Order`
Export (CSV/PDF)	GET `/api/orders/export`, Apache POI oder iText
Bestellstatus	Enum `Status { OFFEN, BEARBEITET, VERSANDT }` in `Order`

Reflexion – KI-Dokumentation

Was war ungenau / wo wurde nachgebessert?

Bereich	Problem (KI-Ausgabe)	Nachbesserung
Schichtentrennung	Preisberechnung im Controller	In `@PrePersist` der Entity verschoben
Validierung	`@Min` für Dezimalpreise	`@DecimalMin("0.01")` ergänzt
CORS	`@CrossOrigin` am Controller	Zentral in `SecurityConfig`
Token-Refresh	Nicht implementiert	`keycloak.updateToken(30)` im Interceptor
H2-Konsole	Durch Security blockiert	`frameOptions.disable()` ergänzt
PUT-Endpunkt	Initial nicht vorhanden	Manuell nachgebaut
Test-Imports	Fehlende `static`-Imports	`import static org.mockito.Mockito.*` vereinfacht

Phase 4 – Testfälle

Die Testfälle wurden zunächst von der KI vorgeschlagen und anschließend fachlich geprüft, ergänzt und korrigiert.

Backend-Tests (automatisiert, 21 Tests gesamt)

Testfall-Tabelle – Controller (HTTP-Ebene)

#	Testfall	Eingabe	Erwartetes Ergebnis	Tatsächl Ergebni
1	Normale Werte	`produktname="Laptop"`, `preis=999.99`, `menge=2`	HTTP 200, `gesamtpreis=1999.98`	HTTP 20
2	Menge = 0	`produktname="Stift"`, `preis=1.50`, `menge=0`	HTTP 400 Bad Request	HTTP 40
3	Menge negativ	`produktname="Stift"`, `preis=1.50`, `menge=-5`	HTTP 400 Bad Request	HTTP 40
4	Preis = 0.00	`produktname="Stift"`, `preis=0.00`, `menge=1`	HTTP 400 Bad Request	HTTP 40
5	Preis negativ	`produktname="Stift"`, `preis=-9.99`, `menge=1`	HTTP 400 Bad Request	HTTP 40
6	Name leer `""`	`produktname=""`, `preis=5.00`, `menge=1`	HTTP 400 Bad Request	HTTP 40
7	Name fehlt (null)	Kein `produktname`-Feld im JSON	HTTP 400 Bad Request	HTTP 40
8	Sehr große Zahlen	`preis=9999999.99`, `menge=99`	HTTP 200, korrekte Summe	HTTP 20
9	Falscher Typ	`einzelpreis="abc"` (String statt Zahl)	HTTP 400 Bad Request	HTTP 40
10	Bestellung löschen	DELETE `/api/orders/1`	HTTP 204 No Content	HTTP 20
11	Gesamtsumme	GET `/api/orders/summe`	HTTP 200 + `gesamtsumme: 149.97`	HTTP 20

Testfall-Tabelle – Gesamtpreis-Berechnung (Unit-Ebene)

#	Testfall	Einzelpreis	Menge	Erwarteter Gesamtpreis	Tatsächlich
1	Normale Werte	9.99 €	3	29.97 €	29.97 € ✓
2	Menge = 1	49.99 €	1	49.99 €	49.99 € ✓

#	Testfall	Einzelpreis	Menge	Erwarteter Gesamtpreis	Tatsächlich
3	Sehr großer Preis	99.999,99 €	1.000	99.999.990,00 €	korrekt ✓
4	Sehr große Menge	0,01 €	1.000.000	10.000,00 €	korrekt ✓

Kritische Prüfung der KI-Testvorschläge:

- KI schlug `menge=0` als Integer-Test vor, vergaß aber den `null`-Fall → manuell ergänzt
- KI testete keinen falschen Datentyp (`"abc"` als Preis) → manuell ergänzt
- KI-Tests für sehr große Zahlen fehlten → als Grenzwertanalyse ergänzt

---

Abschlussreflexion

1. Wo hat die KI gut unterstützt?
Die KI hat die grundlegende Projektstruktur korrekt aufgebaut: Schichtentrennung (Controller → Service → Repository), Maven-Konfiguration, Keycloak-JWT-Integration und die React-Komponenten waren sofort lauffähig. Für Boilerplate-Code wie Entity, Repository-Interface und Axios-Interceptor spart man erheblich Zeit. Auch die Testfälle wurden strukturiert und mit sinnvollen Grenzwerten generiert.

2. Wo war sie fehlerhaft oder ungenau?

Mehrere konkrete Probleme traten auf:

- Gesamtpreis wurde im Controller statt in der Entity berechnet (falsche Schicht)
- `@Min` für Dezimalpreise statt `@DecimalMin("0.01")`
- CORS direkt am Controller statt zentral in der Security-Konfiguration
- Kein automatischer JWT-Token-Refresh — im Produktiveinsatz ein kritischer Bug

- H2-Konsole war durch Spring Security blockiert
- Fehlende statische Imports in den Tests — Code kompilierte nicht
- PUT-Endpunkt wurde initial nicht generiert

3. Würdet ihr KI-Code direkt in ein Kundenprojekt übernehmen?

Nein — nicht ohne gründliche Code-Review. KI-Code ist ein guter erster Entwurf, enthält aber regelmäßig subtile Fehler in Sicherheit, Validierung und Architektur. Im Kundenprojekt müsste jede Zeile geprüft werden: Sicherheitslücken, falsche Abstraktionsebenen und fehlendes Error-Handling

können zu Produktionsausfällen oder Datenverlust führen.

4. Wer trägt die Verantwortung für Fehler?

Der Entwickler, der den Code übernimmt. Die KI ist ein Werkzeug — sie kennt nicht den Kontext, die Sicherheitsanforderungen oder die Konsequenzen von Fehlern. Wer Code deployt, trägt die vollständige Verantwortung — unabhängig davon, ob er selbst oder eine KI ihn geschrieben hat. Das gilt erst recht bei Kundenprojekten mit rechtlichen oder datenschutzrechtlichen Anforderungen.

5. Was habt ihr über KI im Entwicklungsprozess gelernt?

KI ist am nützlichsten als Beschleuniger für bekannte Muster — Boilerplate, Konfiguration, Teststruktur. Sie ersetzt kein technisches Verstndnis: Wer die Grundlagen nicht kennt, erkennt auch die Fehler nicht. Der sinnvollste Einsatz ist: KI generiert den Rahmen, der Entwickler prüft, korrigiert und entscheidet. Wie bei einem Junior-Entwickler — man gibt ihm Aufgaben übernimmt aber nicht blind sein Ergebnis.